

# P1517R0

# Contract Requirements for Iterative High-Assurance Systems

Published Proposal, 2019-03-01

**This version:**

<http://wg21.link/p1517r0>

**Author:**

Ryan McDougall <mcdougall.ryan@gmail.com>

**Audience:**

EWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

C++2a Contracts are trying to serve many masters. If we wish to serve the kind of high-assurance systems that are increasingly embedded in our lives, we should carefully consider the minimal requirements needed to achieve primary goals.

## Table of Contents

### 1 Background

- 1.1 High Assurance Systems
- 1.2 Systemic Failure
- 1.3 Iterative Development

### 2 Contract-Checking Statements

- 2.1 Redundant Checking

### 3 Concerns with Working Draft

#### 3.1 Unchecked Statements and Undefined Behavior

#### 3.2 Uncheckable Statements and Optimization

#### 3.3 Conclusion

### 4 References

## § 1. Background

### § 1.1. High Assurance Systems

Every software system is important. If a video game is buggy it means a loss of money or jobs. If a banking system has errors it could mean a lot of money. If a industrial control system has errors it could mean physical harm. C++ is no stranger to these systems [1][2][3][4][5][6]

We define Quality[7] to be surface area where software performance meets requirements, and we define Reliability[8] to be quality over time. Requirements are always within an operational context, so the expected Operational Domain[9][10] is of integral consideration -- for example the control systems that fly a military jet are not the same as those in a remote control toy.

We define Availability[11] to be continued software performance as time and environment change -- often adversely, such as when given bad data, violated assumptions, or novel environment, as frequently happens in real world deployments. As such we consider Failure to be the adversary of availability. In developing high availability assurance 3 basic tenets may be pursued:

1. Independence[12]: individual failure modes are non-overlapping
2. Redundancy[13]: individual failure occurrence isn't system failure
3. Measurement[14][15]: establish operating boundaries with confidence levels

Failures are a result of Defects within the design, implementation, or assembly of the system.

High assurance systems are written to avoid defects, detect defects, continue in presence of defects, report defects, heal defects, and minimize damage from defects -- often within a statistical, process, or even regulatory framework. These abilities are usually required due to a sensitive or critical operational environment, such as with high energy machinery, or because frequencies or populations cause small probabilities become eventualities.

Therefore what is considered a high assurance system, may be precisely defined, and is a product of set of rigid stipulations that exist to "prove" the assembled whole retains these beneficial

properties[16]. A violation of some constituent property may "pass-the-eye test" or may not "look like a problem" locally, but in the systematic analysis may introduce a flaw in confidence or execution.

High assurance system must have at least two pillars:

1. Be a product of a Verifiable Process[17]
2. Be tested broadly in its Operational Domain[10]

## § 1.2. Systemic Failure

Many defects in maturing systems can be discovered in a finite number of reasonable test cases or deduction steps; we can travel across the country, around the world, or even beyond the atmosphere, with such high confidence that it seems normal or routine. Yet matured systems are not free of accident or disaster. If the system is free of obvious defects, any further failure must result from some defect that escaped a straightforward analysis, through some non-obvious mechanism or happenstance -- one that may have been considered far fetched at design or validation time. Engineering history is written in the ink of regrettable oversights, and the examples of loss are too common. [18][19]

Failures can occur from any combination of defects[20]; after designing and testing for individual defects, we must consider how defects may conspire systematically. Wherever we can imagine a systemic failure there is a defect. If we have already fixed the obvious ones, it must necessarily be the unobvious ones to come.

Assurance is a statistical construct; a confidence built from process and testing, not a guarantee. Black swans exist, and when we allow systematic failure as an unlikely possibility, we enable the risk of it becoming eventuality.

## § 1.3. Iterative Development

In the ideal, software has no defects. Yet it is clear that the systems we rely on do contain latent defects at any given moment, sometimes going undetected for years. Moreover to build assurance we must test in a representative operational domain in order to uncover and fix those defects. So we are certain one way or the other that incorrect software will be in operation in some domain at some point in time.

For products that have a definitive release cycle, we can delay release until we have sufficient confidence through staged, progressively representative test phases, and freeze development until another development cycle is necessary. For the kinds of products we are likely to see more of, such as AI and IoT devices, or even cars[21], the development cycle may be rapid enough that releases are frequent or short. We've gotten used to cloud software or video games that are always updating; one

might imagine a hypothetical Robot Dog constantly learning and adapting to its owners, or an IoT refrigerator that learns when to buy milk.

## § 2. Contract-Checking Statements

Contract-Checking Statements are a natural fit for High Assurance systems:

1. avoid defects through documentation
2. avoid defects through static analysis
3. detect defects and fail fast
4. report defects in violation handlers
5. framework for a verification process

### § 2.1. Redundant Checking

High availability systems cannot "fail fast" once outside the test environment; they are designed to handle defect healing or minimization by reverting to some stable state from which to attempt recovery. We can imagine this working as if an exception is thrown and control reaches a catch clause that can handle it.

```
void handle_drone(FlightPath *path) {
    if (path == nullptr) {
        throw flight_error();
    }
    // ...
}

int main() {
    try {
        begin_flight();
    } catch (flight_error&) {
        land_here();
    }
}
```

Since Contract-Checking Statements are not error handling, we have Redundancy and Independence by encoding our critical logic in both systems.

```

void handle_drone(FlightPath *path)
  [[expects <level> : path != nullptr]] {
    if (path == nullptr) {
      throw flight_error();
    }
    // ...
  }
}

```

Redundant Checking requires:

1. Contract-checking statements are somewhat redundant with fail-over logic
2. Contract-checking statements are set to a semantic where process termination does not result outside test environment

## § 3. Concerns with Working Draft

### § 3.1. Unchecked Statements and Undefined Behavior

If the semantic assigned to a contract checking statement would be undefined behavior when the predicate would evaluate to false, and the compiler is allowed to assume the program has no undefined behavior, it is possible for the compiler to assume the predicate is always true, and use that truth value to optimize the program. While nominally a benefit, if we consider that incorrect programs are commonly in operation, and the very kind of software that is most sensitive to defect will have redundant checking, a change in checking semantic may result in a catastrophic loss of redundant error handling.

```

void handle_drone(FlightPath *path)
  [[expects <level> : path != nullptr]] { // unchecked; assume path never
    if (path == nullptr) {                // elided; path is never null
      throw flight_error();                // elided; fallback is removed
    }

    // **oops** ...
  }
}

```

It may be argued that a sane compiler wouldn't elide error checking if it wasn't "sure", or a high assurance system shouldn't be able to trivially change its build semantic to something dangerous, by

relying on "that shouldn't happen" or "seems unlikely" for our system assurance, we currently expose directly in the text of the working draft a vulnerability that allows for systemic failure as a possibility under some defaults.



High assurance systems would require that any optimization step that might elide redundant checking be the non-default option at any build level.

## § 3.2. Uncheckable Statements and Optimization

We wish to enable contract checking statements as a tool to document interfaces to either humans or static analysis tools, as a means of avoiding defects. However some predicates may not be expressible in a form that can be checked or even evaluate -- e.g. if evaluation would change the state of the system being checked, or if the computation was practically infeasible. However if the only semantic available to express this requirement is axiom, which brings with it the baggage of attempted optimization (and removing our redundant checks), we lose an important tool for achieving high assurance as below.

```
void handle_drone(FlightPath *path)
  [[expects <level> : path != nullptr]]
  [[expects <documentation> : path->is_obstacle_free() ]] {
    if (path == nullptr) {
      throw flight_error();
    }
    // ...
  }
```



Whether the word "axiom" is too overloaded with meaning to meet our requirement, or a new word is needed, high assurance systems require a way to express uncheckable statements that are not allowed to be assumed by the compiler for the purposes of optimization.

### § 3.3. Conclusion

High assurance systems have been written satisfactorily before C++2a contract-checking statements, and may continue to be written with the working draft version. However if C++ community wish to serve this important and growing audience, and wish the full breadth of feature be available to those most in need of them, we should strive to deliver the most sound specification possible.

If the standard itself cannot meet industry expectations, at the very least it should defer to implementation defined behavior, so users can approach their vendors with feature requests. The worst outcome would be if some or all of contract-checking statements ended up on a "banned" list in critical systems.

### § 4. References

- [1]: <http://www.prqa.com/military-aerospacedo-178b/>
- [2]: <http://www.stroustrup.com/JSF-AV-rules.pdf>
- [3]: <https://www.youtube.com/watch?v=3SdSKZFoUa8>
- [4]: <https://www.misra.org.uk/MISRACHome/tabid/128/Default.aspx>
- [5]: <https://www.nrc.gov/reading-rm/doc-collections/nuregs/contract/cr6463/>
- [6]: [https://en.wikipedia.org/wiki/VxWorks#Notable\\_uses](https://en.wikipedia.org/wiki/VxWorks#Notable_uses)
- [7]: [https://en.wikipedia.org/wiki/Software\\_quality](https://en.wikipedia.org/wiki/Software_quality)
- [8]: [https://en.wikipedia.org/wiki/Software\\_reliability\\_testing](https://en.wikipedia.org/wiki/Software_reliability_testing)
- [9]: [https://www.nhtsa.gov/sites/nhtsa.dot.gov/files/documents/13069a-ads2.0\\_090617\\_v9a\\_tag.pdf](https://www.nhtsa.gov/sites/nhtsa.dot.gov/files/documents/13069a-ads2.0_090617_v9a_tag.pdf)
- [10]: <https://www.dmv.ca.gov/portal/dmv/detail/vr/autonomous/definitions>
- [11]: [https://en.wikipedia.org/wiki/High\\_availability\\_software](https://en.wikipedia.org/wiki/High_availability_software)
- [12]: <https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/20160005837.pdf>
- [13]: [https://en.wikipedia.org/wiki/Redundancy\\_\(engineering\)](https://en.wikipedia.org/wiki/Redundancy_(engineering))
- [14]: <https://www.semanticscholar.org/paper/Software-Assurance-Measurement-%E2%80%93-State-of-the-Shoemaker/024f4b9ac5f5b4d75df0013567b4245254e33ba1>

[15]: [https://en.wikipedia.org/wiki/Reliability\\_engineering#Reliability\\_testing](https://en.wikipedia.org/wiki/Reliability_engineering#Reliability_testing)

[16]: <https://en.wikipedia.org/wiki/DO-178C>

[17]: [https://en.wikipedia.org/wiki/Failure\\_mode\\_and\\_effects\\_analysis](https://en.wikipedia.org/wiki/Failure_mode_and_effects_analysis)

[18]: <https://spectrum.ieee.org/tech-history/heroic-failures/the-space-shuttle-a-case-of-subjective-engineering>

[19]: <https://arstechnica.com/cars/2018/05/emergency-brakes-were-disabled-by-ubers-self-driving-software-ntsb-says/>

[20]: [https://en.wikipedia.org/wiki/System\\_accident](https://en.wikipedia.org/wiki/System_accident)

[21]: <https://www.tesla.com/support/software-updates>